

TinyURL Code Walkthrough

TinyURL Code Walkthrough

This document explains the Python TinyURL application step by step. The app is a local URL shortener built with Flask, SQLite, Base62 short keys, bearer-token authentication, custom aliases, redirects, click tracking, and a small browser UI.

Big Picture

The main user flow is:

1. A user submits a long URL.
2. The app validates the URL.
3. The app stores it in SQLite.
4. The app generates a short key using Base62.
5. The user opens the short URL.
6. The app looks up the original URL.
7. The app redirects the browser using HTTP 302.

Project Files

```
.
├── app.py
├── requirements.txt
├── tinyurl/
│   ├── __init__.py
│   ├── app.py
│   ├── auth.py
│   ├── base62.py
│   ├── cache.py
│   └── db.py
```

```
|   ├── static/styles.css
|   └── templates/index.html
└── docs/
    ├── API.md
    ├── CODE_WALKTHROUGH.md
    ├── SYSTEM_DESIGN.md
    └── images/tinyurl-home.png
└── tests/test_app.py
```

1. Root Entrypoint: app.py

```
from tinyurl.app import create_app
```

```
app = create_app()
```

```
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

This is the file you run with:

```
python app.py
```

It imports the Flask app factory, creates the app, and starts the local development server at:

```
http://127.0.0.1:5000
```

The root file stays small so the real application logic can live inside the `tinyurl` package.

2. Package Export: tinyurl/__init__.py

```
from tinyurl.app import create_app
```

```
__all__ = ["create_app"]
```

This makes `create_app` available from the package itself:

```
from tinyurl import create_app
```

It is a convenience file for clean imports.

3. Base62 Short Keys: `tinyurl/base62.py`

```
ALPHABET =  
    "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"  
BASE = len(ALPHABET)
```

Base62 uses 62 characters:

- digits 0–9
- lowercase letters a–z
- uppercase letters A–Z

The app converts database integer IDs into compact strings:

```
1  -> 1  
10 -> a  
61 -> Z  
62 -> 10
```

The main function is:

```
def encode_base62(number: int) -> str:
```

It repeatedly divides the number by 62 and uses each remainder to choose a character from the alphabet. This follows the system design recommendation from the PDF because Base62 is short, deterministic, and fast.

The helper:

```
def is_valid_key(value: str) -> bool:
```

checks whether a short key or custom alias contains only Base62 characters.

4. Local Cache: `tinyurl/cache.py`

This file implements an in-memory LRU cache.

```
class LRUCache:
```

The cache stores:

```
short_key -> long_url
```

Example:

```
abc123 -> https://example.com/article
```

The PDF recommends Redis or Memcached for production hot URL lookups. For local development, this app uses an in-process LRU cache so the project stays simple to run.

Important methods:

```
def get(self, key: str):
```

Looks up a key and marks it as recently used.

```
def set(self, key: str, value: str) -> None:
```

Stores a mapping. If the cache is full, the least recently used item is removed.

```
def delete(self, key: str) -> None:
```

Removes a cached URL when the user deletes it.

```
def clear(self) -> None:
```

Clears all cached entries.

The cache uses RLock so cache operations are safer when multiple requests happen close together.

5. Database Layer: `tinyurl/db.py`

This file creates and opens the SQLite database.

The `users` table stores registered users:

```
id
email
password_hash
created_at
```

The `urls` table stores URL mappings:

```
id
short_key
long_url
user_id
clicks
```

```
expires_at  
created_at  
updated_at
```

The most important mapping is:

```
short_key -> long_url
```

The id column is also important because the app uses it to generate Base62 short keys.

The database has unique indexes for duplicate handling:

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_urls_long_url_public  
ON urls(long_url)  
WHERE user_id IS NULL;
```

This prevents duplicate anonymous URLs.

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_urls_long_url_user  
ON urls(user_id, long_url)  
WHERE user_id IS NOT NULL;
```

This prevents duplicate URLs per authenticated user.

Main functions:

```
def init_db(database_path: str) -> None:
```

Creates the database and tables if they do not already exist.

```
def get_connection(database_path: str) -> sqlite3.Connection:
```

Opens a SQLite connection and returns rows as dictionary-like objects.

```
def row_to_dict(row):
```

Converts SQLite rows into normal dictionaries for JSON responses.

6. Authentication: tinyurl/auth.py

This file handles user passwords and bearer tokens.

```
def hash_password(password: str) -> str:
```

Stores a secure password hash instead of the raw password.

```
def verify_password(password_hash: str, password: str) -> bool:
```

Checks whether a login password matches the stored hash.

```
def create_token(user_id: int) -> str:
```

Creates a signed access token containing the user ID.

Clients send the token like this:

Authorization: Bearer <access_token>

```
def load_user_from_token(token: str):
```

Validates the token and loads the user from the database.

```
def current_user_optional():
```

Reads the Authorization header. If a valid token exists, it returns the user. If not, it returns None.

```
def login_required(view):
```

Protects routes that require authentication. If the user is not logged in, the API returns:

```
{"error": "Authentication required"}
```

with HTTP status 401.

7. App Factory: `tinyurl/app.py`

The main app starts with:

```
def create_app(test_config: dict | None = None) -> Flask:
```

This creates and configures the Flask application.

Important config values:

SECRET_KEY

DATABASE

BASE_URL

```
TOKEN_MAX_AGE_SECONDS
```

```
CACHE_SIZE
```

SECRET_KEY signs auth tokens. DATABASE points to SQLite.

TOKEN_MAX_AGE_SECONDS controls token lifetime. CACHE_SIZE controls how many redirect mappings stay in memory.

Then the app initializes the database:

```
init_db(app.config["DATABASE"])
```

and creates the cache:

```
app.url_cache = LRUCache(app.config["CACHE_SIZE"])
```

8. Current User Before Each Request

```
@app.before_request
```

```
def attach_user():
```

```
    g.current_user = current_user_optional()
```

Before every request, Flask checks whether a bearer token was sent. If yes, it stores the user in `g.current_user`. If no, `g.current_user` is `None`.

This lets `/api/shorten` work for both anonymous users and logged-in users.

9. Home Page Route

```
@app.get("/")
```

```
def home():
```

```
    return render_template("index.html")
```

This renders the browser UI from:

```
tinyurl/templates/index.html
```

10. Health Route

```
@app.get("/health")
```

```
def health():
```

```
    return jsonify({"status": "ok"})
```

This confirms the server is running.

Response:

```
{"status": "ok"}
```

11. Register API

```
@app.post("/api/auth/register")
def register():
```

Expected JSON:

```
{
  "email": "person@example.com",
  "password": "password123"
}
```

Steps:

1. Read JSON from the request.
2. Normalize the email.
3. Validate the email.
4. Validate password length.
5. Hash the password.
6. Insert the user into SQLite.
7. Return an access token.

If the email already exists, the app returns HTTP 409.

12. Login API

```
@app.post("/api/auth/login")
def login():
```

Expected JSON:

```
{
  "email": "person@example.com",
  "password": "password123"
}
```


Steps:

1. Read email and password.
2. Find the user by email.
3. Compare the password with the stored password hash.
4. Return a new access token if valid.
5. Return HTTP 401 if invalid.

13. Shorten URL API

```
@app.post("/api/shorten")
def shorten_url():
```

Expected JSON:

```
{
  "long_url": "https://example.com/article?id=1234"
}
```

Optional fields:

```
{
  "custom_alias": "myArticle",
  "expires_at": "2026-12-31T23:59:59Z"
}
```

Steps:

1. Read request JSON.
2. Extract long_url.
3. Extract optional custom_alias.
4. Extract optional expires_at.
5. Check whether the user is logged in.
6. Validate the long URL.
7. Validate the custom alias if present.
8. Validate expiration datetime if present.
9. Check whether the same URL already exists.
10. Return the existing short URL when appropriate.
11. Otherwise create a new URL mapping.
12. Store the mapping in cache.
13. Return the short URL.

This implements the PDF endpoint:

POST /api/shorten

14. List User URLs API

```
@app.get("/api/urls")
@login_required
def list_urls():
```

This requires a bearer token.

Request:

```
GET /api/urls
Authorization: Bearer <access_token>
```

The route verifies the token, gets the current user ID, fetches URLs owned by that user, and returns them as JSON.

15. Get URL Metadata API

```
@app.get("/api/url/<short_key>")
def get_url(short_key):
```

This returns details for a short URL without redirecting.

It includes:

```
short_key
short_url
long_url
clicks
created_at
expires_at
```

16. Delete URL API

```
@app.delete("/api/url/<short_key>")
@login_required
def delete_url(short_key):
```

This deletes a user-owned URL.

Request:

```
DELETE /api/url/myArticle
Authorization: Bearer <access_token>
```

Steps:

1. Verify the user token.
2. Delete the URL only if the current user owns it.
3. Remove the short key from cache.
4. Return HTTP 204.

This matches the optional delete endpoint from the PDF.

17. Redirect Route

```
@app.get("/<short_key>")
def follow_short_url(short_key):
```

Example:

http://127.0.0.1:5000/abc123

Steps:

1. Validate the short key.
2. Check the in-memory cache.
3. If found in cache, increment clicks and redirect.
4. If not found, query SQLite.
5. Return HTTP 404 if the key does not exist.
6. Return HTTP 410 if the URL expired.
7. Store successful lookup in cache.
8. Increment click count.
9. Redirect to the original URL with HTTP 302.

The redirect happens with:

```
return redirect(row["long_url"], code=302)
```

18. URL Validation

```
def validate_long_url(long_url: str) -> str | None:
```

This only accepts URLs with:

http://

https://

Invalid examples:

example.com

ftp://example.com

hello

19. Custom Alias Validation

```
def validate_alias(alias: str) -> bool:
```

Custom aliases must:

- be 3 to 32 characters long
- contain only Base62 characters

Valid:

myArticle

abc123

MyURL99

Invalid:

my-url

hello_world

a

20. Expiration Handling

```
def parse_iso_datetime(value: str):
```

Parses timestamps such as:

2026-12-31T23:59:59Z

```
def is_expired(row: dict) -> bool:
```

Checks whether a URL has expired. Expired URLs return HTTP 410 Gone.

21. Duplicate URL Lookup

```
def find_existing_url(connection, long_url: str, user: dict | None):
```

For logged-in users, duplicate detection checks:

```
WHERE long_url = ? AND user_id = ?
```

For anonymous users, it checks:

```
WHERE long_url = ? AND user_id IS NULL
```

This means anonymous URLs are deduplicated globally, while user-owned URLs are deduplicated per user.

22. Creating a URL Mapping

```
def create_url_mapping(connection, long_url, custom_alias, user, expires_at):
```

Steps:

1. Insert a row into urls.
2. SQLite creates a unique integer id.
3. Use the custom alias if one was provided.
4. Otherwise encode the integer ID with Base62.
5. Update the database row with the short key.
6. Return the full URL row.

Important line:

```
short_key = custom_alias or encode_base62(row_id)
```

23. URL Lookup

```
def lookup_url(short_key: str):
```

This queries SQLite by short key. It is used by the redirect route, metadata route, and cache-miss path.

24. Click Tracking

```
def increment_clicks(short_key: str) -> None:
```

Every successful redirect increments the `clicks` column. This gives basic analytics for each short URL.

25. API Response Shape

```
def url_payload(row: dict) -> dict:
```

This builds the JSON response for URL records. It uses Flask's `url_for` to create the complete short URL:

```
url_for("follow_short_url", short_key=row["short_key"],
        _external=True)
```

So API responses include:

```
http://127.0.0.1:5000/<short_key>
```

not just the short key.

26. Browser UI: `tinyurl/templates/index.html`

The HTML page contains a form with:

- long URL input
- custom alias input
- shorten button

When the user submits the form, JavaScript calls:

```
fetch("/api/shorten", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(payload)
});
```

The UI uses the same API that curl, Postman, or another frontend would use.

If the API succeeds, it displays the short URL as a clickable link.

27. Styling: `tinyurl/static/styles.css`

The CSS controls:

- page background
- typography
- layout width
- form styling
- input styling
- button styling
- mobile responsiveness

The media query:

```
@media (max-width: 640px)
```

makes the form stack nicely on smaller screens.

28. Requirements: `requirements.txt`

```
Flask==3.0.3
```

```
itsdangerous==2.2.0
```

```
Werkzeug==3.0.3
```

Flask is the web framework. Werkzeug provides lower-level web utilities and password hashing helpers. ItsDangerous provides signed token support.

29. Tests: `tests/test_app.py`

The tests use Python's built-in `unittest`.

Each test creates a temporary database so the real local database is not touched.

Test coverage includes:

- shortening and redirecting
- duplicate anonymous URL handling

- user registration
- authenticated URL creation
- listing user URLs
- deleting user URLs

Run tests with:

```
python3 -m unittest discover -s tests
```

30. Documentation Files

README.md explains setup, running locally, curl examples, tests, and configuration.

docs/API.md documents all API endpoints.

docs/SYSTEM_DESIGN.md explains how the app maps to the system design PDF.

docs/images/tinyurl-home.png is the screenshot used in the README.

Complete Request Flow

Example request:

```
curl -X POST http://127.0.0.1:5000/api/shorten \
  -H "Content-Type: application/json" \
  -d '{"long_url":"https://example.com/article"}
```

Flow:

1. Request reaches Flask.
2. Flask calls shorten_url().
3. The app validates https://example.com/article.
4. The app checks if it already exists.
5. The app inserts into SQLite.
6. SQLite creates id = 1.
7. The app converts 1 to Base62.
8. The short key becomes 1.
9. The app updates the database row.
10. The app stores 1 -> https://example.com/article in cache.
11. The app returns the short URL.

Then the user opens:

`http://127.0.0.1:5000/1`

Flow:

1. Flask calls `follow_short_url("1")`.
2. The app checks cache.
3. The app finds the original URL.
4. The app increments click count.
5. The app returns HTTP 302.
6. The browser goes to `https://example.com/article`.

That is the complete local TinyURL workflow.